

```

    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <scope>runtime</scope>
    <optional>true</optional>
</dependency>

```

Step 2: Develop the domain classes

This project has only one domain class, namely, Question, which is mapped as a JPA entity.

```

@Entity
public class Question {
    @Id
    @GeneratedValue
    private int qid;
    private String description;
    private String optionOne;
    private String optionTwo;
    private String optionThree;
    private String optionFour;
    private int answer;
    private String subject;
    private String topic;

    // add the constructors

    public int getQid() {
        return qid;
    }

    public String getDescription() {
        return description;
    }

    public String getOptionOne() {
        return optionOne;
    }

    public String getOptionTwo() {

```

```

        return optionTwo;
    }

    public String getOptionThree() {
        return optionThree;
    }

    public String getOptionFour() {
        return optionFour;
    }
}

```

It does not offer any setters, as the questions are not expected to be updated once added. Also, since the Question objects will be returned to the REST clients, hide the subject, topic and answer fields by not exposing the corresponding getters.

Step 3: Develop the data layer

The repository follows the JPQL and Spring Data conventions.

```

@Repository
public interface QuestionRepository extends
    JpaRepository<Question, Integer> {
    List<Question> findByQidIn(List<Integer> qids);

    List<QuestionNumber> findQidBySubjectAndTopic(String
subject, String topic);

    long countByQidAndAnswer(int qid, int option);
}

```

The *findByQidIn()* method fetches the questions with the supplied list of Question IDs.

The *findQidBySubjectAndTopic()* method fetches the list of IDs of all the questions belonging to the specified subject and topic. The QuestionNumber interface is used by this method, as the operation involves projection.

```

public interface QuestionNumber {
    public int getQid();
}

```

The *countByQidAndAnswer0* method returns 1 if the supplied option matches the expected answer; otherwise, it returns 0.

Step 4: Develop the REST layer

This layer consists of controllers and DTOs. The QuizController with the *@RestController* annotation maps the REST endpoints to the methods and processes the HTTP requests with the help of injected QuestionRepository.